

Solver Description

Rens Versnel ✉

Utrecht University, The Netherlands

Abstract

Submission to the exact track of the PACE challenge 2026. This solver consists of two parts: a depth bounded search akin to existing FPT algorithms [3] for this problem, and a (original) depth first search procedure that iterates all agreement forests of the instance using incompatible triple and path pair constraints.

2012 ACM Subject Classification Theory of computation → Parameterized complexity and exact algorithms

Keywords and phrases Maximum Agreement Forest, PACE Challenge

1 Short algorithm description

The main algorithm starts by applying common cherry reduction 26, single-vertex tree reduction 27, reduction rule 2.2.1 (from [3]), reduction rule 1 (from [3]), subtree cluster reduction 30, and LSI cluster reduction 29; all exhaustively. Then on each reduced cluster it applies whichever of the two algorithms (AF-DFS and HP-DBS) it deems the most able to solve it, often running AF-DFS for a short period before defaulting to HP-DBS. The AF-DFS algorithm iterates the agreement forests of an instance, by starting at the trivial solution (only single-vertex trees) and then walking through the set of agreement forests while incrementally checking the possible additions against pre-calculated constraints. This makes it very fast if an instance has few agreement forests (which is usually the case when the MAF is very large compared to the number of leaves). The HP-DBS algorithm is more like traditional FPT algorithms, except that it stores a set of ‘hitting pairs’ as constraints, meaning that for each pair one of the two must become a single-vertex tree. This set of hitting pairs helps formulate a few reduction rules and helps in choosing the right branching rule to apply.

Now we will describe the two algorithms separately:

1.1 Agreement forest depth first search (AF-DFS)

There are instances of MAF where the optimal solution has an order nearly as high as the number of leaves; these instances are very hard to solve with depth bounded search algorithms, but should actually not be that hard because there are very few agreement forests that need to be considered. To cover these cases, we introduce a new algorithm that essentially iterates all possible agreement forests in depth first search manner, starting with the trivial solution consisting of only SVTs.

The first step is to define a canonical representation of each agreement forest and describe a DFS tree visiting all representations.

► **Definition 1** (Canonical representation (CANAF)). *Let A be an agreement forest of an instance Ω . For each component of A with at least 2 labels, list its labels in increasing order. Then order these lists in increasing order by their first (smallest) label. The resulting ordered collection $S_1, S_2, \dots, S_m$ is the canonical representation of A ($CANAF(A)$).*

We define two operations on the canonical representations to enumerate all possible representations:

► **Definition 2** (Extend last component (EXTENDLASTCOMP)). Let $S_1, S_2, \dots, S_m$ be a canonical representations of an agreement forest A of instance Ω . Let t_{max} be the last (largest) element of S_m and let T be the set of SVTs in A (i.e., the labels present in A , but not in $\bigcup_{i=1}^m S_i$) that are larger than t_{max} . The procedure $EXTENDLASTCOMP(A, \Omega)$ iterates all canonical representations created by appending a label from T to S_m and returns the subset of these that is an agreement forest of Ω .

► **Definition 3** (Add new component (ADDNEWCOMP)). Let $\mathcal{S} = S_1, S_2, \dots, S_m$ be a canonical representations of an agreement forest A of instance Ω . Let t_{min} be the first (smallest) element of S_m and let T be the set of SVTs in A (i.e., the labels present in A , but not in $\bigcup_{i=1}^m S_i$) that are larger than t_{min} . The procedure $ADDNEWCOMP(A, \Omega)$ iterates all canonical representations created by appending component to $\mathcal{S}$ consisting of two different labels from T in sorted order, and returns the subset of these that is an agreement forest of Ω .

Note that we will apply $EXTENDLASTCOMP$ and $ADDNEWCOMP$ on an agreement forest or on the canonical representation of that agreement forest, which will mean the exact same due to the instance Ω through which the canonical representation can be translated to an agreement forest and vice versa. Likewise, we may assume the operations return agreement forests or their canonical representations. We will show how the canonical representations relate to the order of their agreement forests:

► **Lemma 4.** Let $\mathcal{S} = S_1, S_2, \dots, S_m$ be a canonical representation of an agreement forest A with n labels/leaves. Then, $ORD(A) = n - \sum_{i=1}^m (|S_i| - 1)$.

Proof. We will prove this by induction on $|\mathcal{S}|$: For the base case $|\mathcal{S}| = 0$ (i.e., $\mathcal{S} = \emptyset$) it is clear that this holds, because all n labels a single vertex trees. Then for the inductive step: let $\mathcal{S}$ be a canonical representation of A with $|\mathcal{S}| = s$ and assume the lemma holds for all $\mathcal{S}'$ with $|\mathcal{S}'| < s$. Remove the last element S_m from $\mathcal{S}$ to obtain representation $\mathcal{S}^* = S_1, \dots, S_{m-1}$ of agreement forest A^* with $|\mathcal{S}^*| = s - 1$. It is clear that exactly $|S_m|$ labels that formed one component before have now become SVTs instead. This increases the order of the corresponding agreement forest by $|S_m| - 1$. Then it follows that $ORD(A) = ORD(A^*) - (S_m - 1) = n - \sum_{i=1}^{m-1} (|S_i| - 1) - (S_m - 1) = n - \sum_{i=1}^m (|S_i| - 1)$. ◀

As a result, it is also clear that both operations $EXTENDLASTCOMP$ and $ADDNEWCOMP$ return representations of agreement forests with an order of exactly one more than their input agreement forest.

To show that we will iterate all agreement forest of an instance, we will first make an two observations:

► **Observation 5.** Let A be an X -forests and Ω and instance on label-set X and let B be an X -forest obtained from A by cutting the edge above one leaf, making its label into a SVT. If A is an agreement forest of Ω , then so is B .

► **Observation 6.** Every agreement forest A of Ω , except for the trivial one, has some X -forest A' such that $EXTENDLASTCOMP(A', \Omega)$ or $ADDNEWCOMP(A', \Omega)$ produces A .

To be convinced of Observation 6, we can either remove the last label of the last component of $CANAF(A)$ if it has at least three labels, or otherwise remove the last component entirely. The result is a canonical representation of the desired A' .

Cutting one leaf is the inverse operation of $EXTENDLASTCOMP$ and $ADDNEWCOMP$, so as a consequence of Observation 5, the X -forest A' from Observation 6 is also an agreement forest for Ω . This shows the correctness of the following corollary:

```

1 Procedure ITER-AF-DFS( $\Omega, A$ )
   Input: A set  $\Omega = \{F_1, \dots, F_t\}$  of  $X$ -forests and an agreement forest  $A$  of
           instance  $\Omega$ .
   Output: A set of agreement forests of  $\Omega$  reachable from  $A$  by
           EXTENDLASTCOMP or ADDNEWCOMP operations.
2    $\mathcal{A} \leftarrow \text{EXTENDLASTCOMP}(A, \Omega)$ 
3    $\mathcal{B} \leftarrow \text{ADDNEWCOMP}(A, \Omega)$ 
4   return  $\mathcal{A} \cup \mathcal{B} \cup \{A\}$ 

5 Procedure AF-DFS( $\Omega$ )
   Input: A set  $\Omega = \{F_1, \dots, F_t\}$  of  $X$ -forests.
   Output: An minimum order agreement forest for  $\Omega$ .
6    $A \leftarrow$  the trivial agreement forest of  $\Omega$ , consisting of SVTs
7    $\mathcal{A} \leftarrow \text{ITER-AF-DFS}(\Omega, A)$ 
8   return the agreement forest in  $\mathcal{A}$  with minimum order

```

■ **Figure 1** An exact algorithm for computing a MAF

► **Corollary 7.** *Every agreement forest A of an instance Ω with n leaves with $\text{ORD}(A) < n$ has some agreement A' with $\text{ORD}(A') = \text{ORD}(A) + 1$ such that $\text{EXTENDLASTCOMP}(A', \Omega)$ or $\text{ADDNEWCOMP}(A', \Omega)$ produces A .*

Now we are ready to formulate the depth first search routine iterating all agreement forests and show its correctness:

► **Lemma 8.** *$\text{AF-DFS}(\Omega)$ is an exact algorithm for $\text{MAF}(\Omega, k)$.*

Proof. Let Ω be an instance with n labels. We will show that *ITER-AF-DFS* iterates all agreement forests of Ω after which the correctness follows immediately. First note that the trivial agreement forest with which *AF-DFS* starts is of order n and it is the only agreement forests of Ω with order n or higher. This forms the base case for our induction. For the inductive step, we assume *ITER-AF-DFS* returns all agreement forests of order at least $k + 1$. Let A be an agreement forest of Ω of order k . From Corollary 7 it follows that there is some agreement forest A' of order $k + 1$ that produces A by an application of *EXTENDLASTCOMP* or *ADDNEWCOMP*, thus using by the inductive hypothesis we know all agreement forests of order k are also returned by *ITER-AF-DFS*. Then, by induction we have shown that all agreement forests of Ω are returned by the *ITER-AF-DFS* procedure and thus *AF-DFS* finds a MAF, which is clearly enough to solve the decision variant, $\text{MAF}(\Omega, k)$. ◀

Determining if an X -forest is an agreement forest

To complete the description, we will describe how we determine if an X -forest or its canonical representation is an agreement forests for an instance Ω (which we use in the *EXTENDLASTCOMP* and *ADDNEWCOMP* operations). Although it should be clear that it can be done in polynomial time, we will describe one of the options for the sake of completeness.

The approach is based on an ILP formulation for the rSPR distance problem, and thus for MAF on two binary trees, by Wu [4], which we extended to solve MAF on multiple binary trees. Wu defines two types of constraints, incompatible triple constraints and incompatible path constraints, which we will describe a modified version of:

► **Definition 9** (Incompatible triple). *Let Ω be an instance of MAF and let a, b and c be three distinct labels from Ω . We say triple $\{a, b, c\}$ is an incompatible triple if there is a pair of forests F_a, F_b in Ω in which a, b and c are in the same component and for which the subgraphs induced by $\{a, b, c\}$ are not the same.*

► **Definition 10** (Incompatible path pair). *Let Ω be an instance of MAF and let a, b, c and d be four distinct labels from Ω . We say the collection $\{\{a, b\}, \{c, d\}\}$ is an incompatible path pair if there is a forests F in Ω in which a and b are connected and c and d are connected and in which the path between a and b shares an edge with the path between c and d .*

► **Lemma 11.** *An X -forest A on the label-set of an instance Ω is an agreement forest for Ω if and only if:*

1. *no incompatible triple of Ω has all three labels in the same component in A , and*
2. *there is no incompatible path pair $\{\{a, b\}, \{c, d\}\}$ where a and b are connected and c and d are connected in A , and a and c are not connected in A .*

Proof. $\Rightarrow$: Let X -forest A be an agreement forest for Ω . We will show that conditions (1) and (2) hold for agreement forest A :

1. Suppose for contradiction that there exists an incompatible triple $\{a, b, c\}$ with all three labels in the same component in A_i in A . Since A is an agreement forest for each X -forest F_j in Ω , it is by definition a subforest of each F_j and so is the component A_i . It is clear that, because a, b and c are all in A_i , the induced subgraph of $\{a, b, c\}$ in A_i is the same as any X -forest that A is a subforest of. Thus all X -forests in Ω must also have the same induced subgraph of $\{a, b, c\}$ (or have them separated, which immediately contradicts A being their agreement forest). This contradicts the definition of an incompatible triple, thus no such incompatible triple can exist.
2. Suppose for contradiction that there is an incompatible path pair $\{\{a, b\}, \{c, d\}\}$ where a and b are connected and c and d are connected in A , and a and c are not connected in A . Let A_1 and A_2 be the components of a and b and of c and d respectively. Let $F \in \Omega$ be an X -forest where $\{\{a, b\}, \{c, d\}\}$ is an incompatible triple. Because A is a subforest of F , all edges on the path between a and b must be in A and likewise all edges between c and d must be in A . Since these paths overlap in F , the edges on the path between a and c are a subset of these paths between a, b and between c, d , and thus must also be in A . Since all edges between a and c are in A , they must be in the same component in A contradicting our assumption that such an incompatible path pair exists.

◀

$\Leftarrow$: Let F be an X -forest in Ω and let A be another X -forest satisfying conditions (1) and (2). We will show that A is an agreement forest for F by showing it is a subforest of F : For each component A_i of A , let $X_i \subseteq X$ be its label-set, and let $F(X_i)$ be the minimal subtree of F spanning X_i (i.e. all edges between pairs of labels in X_i).

First, the subtrees $F(X_i)$ must be edge-disjoint in F . Suppose for contradiction that $F(X_i)$ and $F(X_j)$ share an edge e for some $i \neq j$. Then there exist leaf pairs $a, b \in X_i$ and $c, d \in X_j$ whose path in F share edge e . By definition, $\{\{a, b\}, \{c, d\}\}$ is an incompatible path pair that violates Condition (2).

Second, each A_i is isomorphic to $F(X_i)$ up to forced contraction. By the well know lemma by Bordewich and Semple [2] two rooted binary trees are equivalent iff every triple of leaves is topologically equivalent. Thus by Condition (1) A_i is isomorphic to $F(X_i)$ for each i .

Then removing all edges that do not belong to any $F(X_i)$ from F results in a subforest F' , because all $F(X_i)$ are edge-disjoint. This subforest is isomorphic to A because every

component in it is isomorphic to a unique component in A and vice versa. Thus A is a subforest of F and therefore an agreement forest. $\blacktriangleleft$

Note that also create a DP table containing for each component A_i all extensions labels a such that putting a at the end of A_i results is a valid component w.r.t the incompatible triple constraints. To create this table we start off with a set of *compatible* triples C_3 and for each compatible triple (a, b, c) one we iterate each label d that is larger than c . For each d , we check if all subsets of size 3 of (a, b, c, d) are compatible triples and if so we add d to $E((a, b, c))$ and we add (a, b, c, d) to C_4 . Now C_4 contains all compatible quads and we can perform the same steps to calculate the possible extensions for this set, etc. The result is a map E from valid components to valid extensions w.r.t incompatible triple constraints.

The proof that this is valid is simple: if any component D in C_i is not valid, it must contain an incompatible triple, and thus has some subset in $C_{(i-1)}$ that contains it, contradicting the fact that D can ever be created.

1.2 Hitting pair depth bounded search (HP-DBS)

This depth bounded search algorithm is similiary to

► **Definition 12** (Hitting Pair). *Let X be a set of labels. A hitting pair set H is a set of pairs of two different labels, i.e., $H \subseteq (X \times X) \setminus \{\{x, x\} | x \in X\}$.*

The idea of a hitting pair $\{a, b\}$ is that it forces at least one of a and b to be a single vertex tree in the current execution branch. In this algorithm, whenever we consider a set of hitting pairs H for forests Ω , we will assume that all hitting pairs are pairs of labels that form a cherry in at least one forest in Ω ; it is easy to verify that this constraint holds whenever a reduction or branching rule adds a hitting pair. Throughout this description we will also assume that whenever a label is made a single vertex tree in all forests all hitting pairs containing that label are discarded.

The modified maximum agreement forest problem that incorporates these hitting pairs and their constraints is formulated below:

Hitting Pair Maximum Agreement Forest problem (HP-MAF)

INPUT: A set $\Omega = \{F_1, \dots, F_t\}$ of X -forests and a set $H \subseteq (X \times X) \setminus \{\{x, x\} | x \in X\}$ of hitting pairs (informally, a set of pairs of two different labels) and a parameter k .

OUTPUT: An agreement forest for Ω of order at most k , in which for each hitting pair $\{a, b\} \in H$ either a is a single vertex tree or b is, or both, or report that no such agreement forest exists.

In our proofs we will also use the decision variant of this problem $\text{HP-MAF}_D(\Omega, H, k)$ that returns “true” if $\text{HP-MAF}(\Omega, H, k)$ finds an agreement forest and “false” otherwise.

The common cherry reduction 26 and single vertex tree reduction 27 rules are applied exhaustively between all other branching or reduction rules. The single vertex tree reduction rule is virtually identical to the base version 27, except that hitting pairs containing a single vertex tree can be forgotten; however, the common cherry reduction rule requires some modification to adhere to the hitting pair constraints:

► **Definition 13** (HP-Single Vertex Tree Reduction (HP-RR1)). *Let (Ω, H, k) be an instance. If some label a forms a single vertex tree in some forest in Ω , make a a single vertex tree in every forest in Ω .*

It should be evident that with the introduction of hitting pairs the proof of correctness for the Single Vertex Tree Reduction 27 is still sufficient: if a label is a single vertex tree in any forest in Ω , then all agreement forests of Ω must have that same label as a single vertex tree, in particular the *maximum* agreement forests of Ω and in particular also the agreement forests satisfying the constraints of the hitting pairs.

► **Definition 14** (HP-Common Cherry Reduction (HP-RR2)). *Let (Ω, H, k) be an instance. If two labels a and b form a cherry common to all forests in Ω then perform one of two reductions:*

- *If $\{a, b\} \in H$, make either a or b a single vertex tree in all forests.*
- *Otherwise, in each forest, merge the leaves with labels a and b into their parent v , giving v a new (arbitrary) label m_{ab} . To obtain a solution for the original instance, simply replace m_{ab} by an inner node with a and b as children in each forest.*

► **Lemma 15.** *HP-Reduction Rule 2 is safe.*

Proof. We will prove this for the two rules in HP-RR2 separately. First let (Ω, H, k) be an instance where the first rule of HP-RR2 is applicable on a common cherry with labels a and b that is also a hitting pair in H and let (Ω', H', k) be the result of this application. We will show that $\text{HP-MAF}_D(\Omega, H, k) \Leftrightarrow \text{HP-MAF}_D(\Omega', H', k)$.

⇐: This direction follows trivially: any agreement forest of Ω' is necessarily an agreement forest of Ω . ◀

⇒: Let agreement forest A be a solution to $\text{HP-MAF}(\Omega, H, k)$. By the constraints of hitting pairs, either a or b must be SVT in A . But we can even show that a modification of A , say A' , in which the labels a and b are swapped is also a solution. This holds because before the application of HP-RR2 the labels a and b are in equivalent positions in each forest (i.e., they can be swapped around without changing the solution space) and due to the constraints on hitting pairs $\{a, b\}$ is the only hitting pair containing a or b . In conclusion, for both a and b a solution to $\text{HP-MAF}(\Omega, H, k)$ exists where it is SVT and thus that solution is also a solution to $\text{HP-MAF}(\Omega', H', k)$. ◀

Now let (Ω, H, k) be an instance where the second rule of HP-RR2 is applicable on a common cherry with labels a and b and let (Ω', H', k) be the result of this application (note that H and k remain unchanged after the reduction). We will show that $\text{HP-MAF}_D(\Omega, H, k) \Leftrightarrow \text{HP-MAF}_D(\Omega', H, k)$.

⇐: Let A' be a solution to $\text{HP-MAF}(\Omega', H, k)$. We can construct a solution to $\text{HP-MAF}(\Omega, H, k)$ by “unmerging” the leaf with label m_{ab} , i.e., replacing it by an inner node with leaves with labels a and b as children. ◀

⇒: Let A be a solution to $\text{HP-MAF}(\Omega, H, k)$ with minimum order. In A , the labels a and b must form a cherry, otherwise one of the two must be SVT, say a , and we can construct a smaller solution by attaching a to the middle of the edge above b (or making a and b the children of a new root if b was also SVT). Because a and b form a cherry in A we can merge them into $m_{a,b}$ to obtain a solution to $\text{HP-MAF}(\Omega', H, k)$. ◀

In the following, we will say an instance is *RR1-reduced* (respectively *RR2-reduced*) if HP-RR1 (resp. HP-RR2) is applied exhaustively.

The introduction of hitting pairs also allows us to formulate a modified version of the base branching rule 2.1 as defined in [3]. Let (Ω, H, k) be an *RR1-* and *RR2-reduced* instance and let $\{a, b\}$ be a cherry in some forest in Ω where $\{a, b\} \notin H$.

► **Definition 16** (HP-Branching Rule 1 (HP-BR1)). *Branch in two ways:*

- *Remove all subtrees pendant to the path between a and b in all forests;*

■ Add $\{a, b\}$ to H .

► **Lemma 17.** *HP-Branching Rule 1 is safe.*

Proof. Let (Ω, H, k) be an instance where HP-BR1 is applicable on labels a and b . Let A be a solution to HP-MAF(Ω, H, k). To prove that the branching rule is safe it suffices to show that in A either label a or b is a single vertex tree or they form a cherry (i.e., there are no subtrees pendant to the path between a and b in any forest). We assume a and b are not single vertex trees in A , otherwise we are done. Then it follows that a and b are in the same component in A because in some forest in Ω they form a cherry and the only way to separate them is by making either a single vertex tree. Then, since a and b form a cherry in one forest, they must do so in A as well. ◀

The previous branching rule is not applicable when all cherries in the instance are already a hitting pair. To complement this we introduce another branching rule that uses these hitting pairs.

Let (Ω, H, k) be an $RR1$ - and $RR2$ -reduced instance and let a be a label that is in at least one hitting pair and let B be the set of labels that is in a hitting pair with a (i.e., $B = \{b \mid \{a, b\} \in H\}$).

► **Definition 18** (HP-Branching Rule 2 (HP-BR2)). *Branch in two ways:*

- Make a a single vertex tree in all forests;
- Make all labels in B single vertex trees in all forests.

► **Lemma 19.** *HP-Branching Rule 2 is safe.*

Proof. Let (Ω, H, k) be an instance where HP-BR2 is applicable label a and set of labels B . Let A be an agreement forest of Ω and a solution to HP-MAF(Ω, H, k). To prove that the branching rule is safe it suffices to show that in A either label a is a single vertex tree or all labels in B are. This follows directly from the constraints of the hitting pairs: if a is not SVT, then the constraints force every label in B to be SVT. ◀

Note that in any $RR1$ - and $RR2$ -reduced instance either HP-BR1 or HP-BR2 is applicable or it is completed (i.e., no forest contains a cherry, and thus all forests are the same collection of single vertex trees), because otherwise H is empty which means there are no cherries in Ω otherwise $HP - BR1$ is applicable, and thus the instance consists of only single vertex trees and is completed.

Note that an application of branching rule 2.1 from [3] can be simulated by consecutively applying HP-BR1 and HP-BR2 on the same label pair (when starting with an empty hitting pair set). However, the advantage of splitting this into two rules is that more information (i.e., hitting pairs) can be gathered before branching which can be used for approximations leading to earlier pruning or can be used for smarter branching choices reducing the practical branching factor. For example, take an instance where HP-BR1 was applied three times on cherries $\{a, b\}$, $\{a, c\}$ and $\{a, d\}$ which are very far apart in some forest(s), leading to the first branching call (removing all pendant subtrees) to terminate early; then we are in the latter case in these three HP-BR1 applications and this instance has three hitting pairs and applying HP-BR2 will result in either making a a single vertex tree or making b, c , and d single vertex trees.

Despite the introduction of hitting pairs, reduction rule 2.2.1 from [3], which can have a great impact in some cases, is still applicable in most cases with some modifications:

► **Definition 20** (HP-Reduction Rule 3). *Let (Ω, H, k) be an RR1- and RR2-reduced instance and a and b be two labels that form a cherry in some forest and are not in any hitting pairs in H (i.e., $\nexists c$, s.t. $\{a, c\} \in H \vee \{b, c\} \in H$). If in every forest:*

- *a and b are either a cherry or separated by only one pendant subtree from becoming a cherry (i.e., there are 0 or 1 pendant subtrees on the path between a and b);*
 - *and the label-sets of this pendant subtree are the same across all forest,*
- then cut the pendant subtree between a and b (if applicable) in each forest, thereby making a and b a cherry in each forest.*

The proof of this reduction rule is also a slightly modified version of the proof given Lemma 4.4 in [3]:

► **Lemma 21.** *HP-Reduction Rule 3 is safe.*

Proof. Let (Ω, H, k) be an instance where HP-RR3 applicable on labels a and b with label-set C below the pendant subtrees and let (Ω', H', k) be the result of this application. We will show that $\text{HP-MAF}_D(\Omega, H, k) \Leftrightarrow \text{HP-MAF}_D(\Omega', H', k)$.

⇐: This direction follows trivially: any agreement forest of Ω' is necessarily an agreement forest of Ω and the only hitting pairs in $H - H'$ are those that were satisfied by performing the reduction. ◀

⇒: Let agreement forest A be a solution to $\text{HP-MAF}(\Omega, H, k)$. If a and b are in the same component of A , they must be siblings because they are siblings in some forest in Ω , which immediately makes A a solution to $\text{HP-MAF}(\Omega', H', k)$ as well. Thus, we assume that a and b are not in the same component, which forces at least one of a and b to be SVT. Without loss of generality, we assume that a is a single vertex tree in A . There are two cases for b :

- Label b is a single vertex tree in A . We can assume there is a component in A that contains labels from both C and $X \setminus \{a, b\} \setminus C$, otherwise joining a and b already forms a solution to $\text{HP-MAF}(\Omega', H', k)$. Because there can only be one such component we can split it into the part with labels from C and the rest to obtain an agreement forest A' with $\text{ORD}(A') = \text{ORD}(A) + 1$. Now labels a and b can be joined in A' to obtain a new agreement forest A^* in which a and b are a cherry, which is a solution to $\text{HP-MAF}(\Omega', H', k)$.
- Label b is not a single vertex tree in A . Again, we assume there is a component in A that contains labels from both C and $X \setminus \{a, b\} \setminus C$, otherwise attaching a to the middle of the edge above b forms a solution to $\text{HP-MAF}(\Omega', H', k)$. Then using the argument above, we can split this component into two and then attach a to b to obtain a solution A^* to $\text{HP-MAF}(\Omega', H', k)$.

Note that the only labels that are SVT in A and not in A^* are a and b which are assumed not to be part of any hitting pair in H (and H'); thus the hitting pair constraints are still satisfied. ◀

To generate more hitting pairs, a new reduction rule is introduced. The idea behind it is the same as a well-known branching rule that states that if labels a and b form a cherry in one forest and are in different components in another, then (at least) one of them must become a single vertex tree.

► **Definition 22** (HP-Reduction Rule 4 (HP-RR4)). *Let (Ω, H, k) be an RR1- and RR2-reduced instance. If two labels a and b form a cherry in some forest and are in different components in another forest, then add hitting pair $\{a, b\}$ to H .*

► **Lemma 23.** *HP-Reduction Rule 4 is safe.*

Proof. Let (Ω, H, k) be an instance where HP-RR4 is applicable on labels a and b and let (Ω', H', k) be the result of this application. We will show that $\text{HP-MAF}_D(\Omega, H, k) \Leftrightarrow \text{HP-MAF}_D(\Omega', H', k)$.

$\Leftarrow$: This direction follows trivially: any agreement forest of Ω' is necessarily an agreement forest of Ω and the only hitting pairs in $H - H'$ are those that were satisfied by performing the reduction. $\blacktriangleleft$

$\Rightarrow$: Let agreement forest A be a solution to $\text{HP-MAF}(\Omega, H, k)$. If neither a nor b is SVT, then they must be in the same component in A , because the only way to disconnect a and b in a forest where they are siblings is by making either SVT. However, this results in a contradiction, because there was some other forest where a and b are in different components, meaning they must also be in other components in A . By contradiction at least one of a and b is SVT in A which means A is also a solution to $\text{HP-MAF}(\Omega', H', k)$. $\blacktriangleleft$

The introduction of hitting pairs also allows for the formulation of another reduction rule:

► **Definition 24** (HP-Reduction Rule 5 (HP-RR5)). *Let (Ω, H, k) be an RR1- and RR2-reduced instance. Let C be the label-set of some component of a forest in Ω with $|C| \geq 2$. If a is a label in C and for every label $c \in C \setminus \{a\}$, $\{a, c\}$ is a hitting pair in H , then make label a a single vertex tree in each forest in Ω .*

► **Lemma 25.** *HP-Reduction Rule 5 is safe.*

Proof. Let (Ω, H, k) be an instance where HP-RR5 is applicable on label a within a component in F with label-set C and let (Ω', H', k) be the result of this application. We will show that $\text{HP-MAF}_D(\Omega, H, k) \Leftrightarrow \text{HP-MAF}_D(\Omega', H', k)$.

$\Leftarrow$: This direction follows trivially: any agreement forest of Ω' is necessarily an agreement forest of Ω and the only hitting pairs in $H - H'$ are those that were satisfied by performing the reduction. $\blacktriangleleft$

$\Rightarrow$: Let agreement forest A be a solution to $\text{HP-MAF}(\Omega, H, k)$. If a is not SVT in A , then by the constraints of the hitting pairs all labels in $C \setminus \{a\}$ are SVT and thus in forest F the label a is the only label left in its component, making it SVT. Then any agreement forest of F must have a as SVT, resulting in a contradiction. Since a is SVT in A , it is also a solution to $\text{HP-MAF}(\Omega', H', k)$. $\blacktriangleleft$

Exact algorithm for Maf

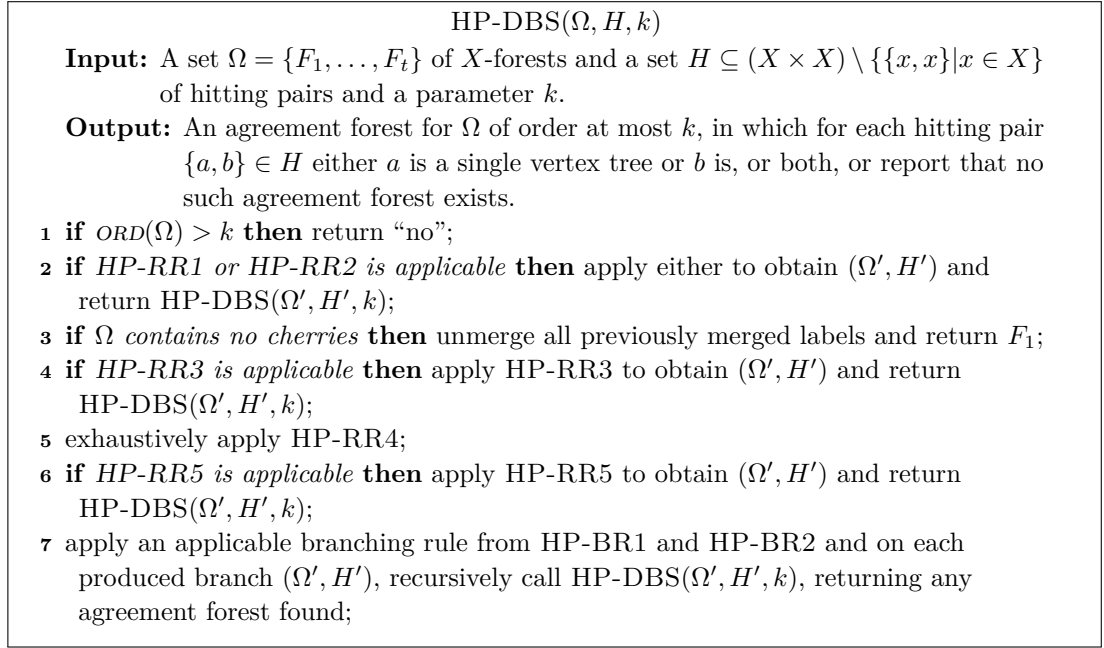
The HP-DBS algorithm for solving HP-MAF is described in Figure 2. It should be clear that HP-DBS is also an exact algorithm MAF: any instance of (Ω, k) of MAF can be solved by solving HP-MAF $(\Omega, \emptyset, k)$ instead.

2 Definitions

► **Definition 26** (Common Cherry Reduction). *Let Ω be an instance. If two labels a and b form a cherry in all X -forests in Ω , then in each forest merge the leaves with labels a and b into their parent v , giving v a new label m_{ab} to obtain a new instance Ω' . Then $|\text{MAF}(\Omega)| = |\text{MAF}(\Omega')|$.*

The correctness of this reduction rule should be clear.

► **Definition 27** (Single Vertex Tree Reduction). *Let Ω be an instance. If a label a is a single vertex tree in any forest in Ω , make it a single vertex tree in every forest in Ω to obtain Ω' . Then $|\text{MAF}(\Omega)| = |\text{MAF}(\Omega')|$.*



■ **Figure 2** An exact algorithm for HP-MAF (Ω, H, k)

The correctness of this reduction rule should be clear.

► **Definition 28** (Label-Set Isomorphism (LSI)). *Two X -forests F and F' satisfy the label-set isomorphism property if for each component C in F , there is a component C' in F' such that $L(C) = L(C')$. A set of X -forests satisfies the LSI property if any two X -forests in the set satisfy the LSI property.*

3 Cluster reduction

There are two types of cluster reduction known for this problem: the first type we will call LSI cluster reduction, and is applicable when an instance consists of multiple separate clusters as the components of its forests, i.e., it satisfies the label-set isomorphism (see the Definition 28). In this case we can decompose the instance, solve the clusters separately and recombine the agreement forests into an optimal one for the main instance.

► **Definition 29** (LSI Cluster Reduction). *Let (Ω, k) be an instance satisfying the label-set isomorphism containing at least two clusters (i.e., $\text{ord}(\Omega) > 1$). Split the instance into its components by repeatedly selecting a component in F_1 with some label-set S and extracting the component with label-set S from each forest to obtain a new instance Ω_i and the reduced main instance Ω . Find the maximum agreement forest of Ω_i and let k_i be its order. Repeat this until Ω is empty. If at the end the sum of all k_i is more than k , report ‘infeasible’, otherwise reconstruct a solution to the original problem by joining the solution forests into one.*

It is easy to see that LSI cluster reduction is safe: a collection of agreement forests of the clusters with total order less than k immediately forms an agreement forest of Ω of the same size. Conversely, if no collection of agreement forests exists for the clusters with total order

less than k , then no agreement forests for Ω with order less than k can exist, because it can be decomposed to create the contradicting collection of agreement forests for the clusters.

We will also say LSI cluster reduction is applicable when an instance only partly satisfies LSI, meaning there is at least one LSI cluster and a part the remains after removing all LSI clusters. In this case we can simply regard the remaining part as another LSI cluster because it is evidently independent from the LSI clusters.

The second type of cluster reduction we will call subtree cluster reduction, and is applicable when all forests have a subtree with the same label-set. It was described recently by Kelk [1] for two input forests, along with sketch proofs and an implementation, but the same logic works for more than two input forests. The idea is to detach the common subtree in each forest, obtaining two separate clusters, and leave a dummy label on each end of the cut edges; if the introduction of this dummy label does not increase the order of the optimal solution, we know that some optimal solution bridges this edge that was cut. Then a solution can be constructed by regrafting these solutions at their dummy labels.

► **Definition 30** (Subtree Cluster Reduction). *Let (Ω, k) be and let S be a label-set such that each forest F_i has a vertex v_i with $L(v_i) = S$. Cut vertex v_i in each forest and extract this subtree to obtain a separate cluster Ω_{below} with label-set S and the remaining cluster Ω_{above} with the rest of the labels. For both clusters, in each forest, at the position where the cut was performed attach a dummy label to obtain two new clusters Ω'_{below} and Ω'_{above} . This dummy label is placed as a new root in Ω'_{above} and is placed at the old locations of v_i in Ω'_{below} . Solve the clusters $\Omega_{\text{below}}, \Omega_{\text{above}}, \Omega'_{\text{below}}$ and Ω'_{above} to optimality and let their solution be A, B, A' and B' respectively. Then perform case distinction on the order of these solutions:*

- *If $\text{ord}(A) + \text{ord}(B) \leq k$: join forests A and B into one and return it.*
- *If $\text{ord}(A) + \text{ord}(B) > k + 1$: report ‘infeasible’*
- *Otherwise, if $\text{ord}(A) = \text{ord}(A')$ and $\text{ord}(B') + \text{ord}(B')$: merge solutions A' and B' into one by removing the dummy labels and attaching the trees containing those them at the previous position of the dummy labels. Then, return this merged solution.*
- *Otherwise: report ‘infeasible’.*

A sketch proof for this in the case of two input forests was recently given by Kelk [1]. The proof below uses the ideas from [1] and extends it to multiple input forests, which requires barely any modifications.

► **Lemma 31.** *Subtree Cluster Reduction is safe.*

Proof. We will show this for each of the four cases separately. Let (Ω, k) be an instance where subtree cluster reduction is applicable on cluster label-set S .

- If $\text{ord}(A) + \text{ord}(B) \leq k$: it is clear that in this case joining the solutions A and B together forms a solution to $\text{MAF}(\Omega, k)$.
- If $\text{ord}(A) + \text{ord}(B) > k + 1$: it is also clear in this case that there is no solution to $\text{MAF}(\Omega, k)$, because it would immediately lead to a contradiction because such a solution can be decomposed into agreement forests of Ω_{above} and Ω_{below} with $\text{ord}(A) + \text{ord}(B) \leq k + 1$.
- Otherwise, if $\text{ord}(A) = \text{ord}(A')$ and $\text{ord}(B') = \text{ord}(B')$: in this case we know $\text{ord}(A) + \text{ord}(B) = \text{ord}(A') + \text{ord}(B') = k + 1$. We also know that in both A' and B' the dummy label is not a single vertex tree, otherwise we could construct an agreement forest of lower order than A or B , contradicting the optimality. The agreement forest A' and B' can clearly be joined to obtain an agreement forest of Ω of order $k + 1$ after removing the dummy labels. However, we can merge the components previously containing the dummy labels, say A'_C for A' and B'_C for B' , into one because in each forest there is the edge

v_i that was cut that separates all labels in A'_C from all in B'_C (in the same orientation). The agreement forest obtained this way is exactly of order k .

- Otherwise: in this case we know $\text{ord}(A) + \text{ord}(B) = k+1$ and thus $\text{ord}(A') + \text{ord}(B') \geq k+2$. To show there cannot be a solution, assume to the contrary that there is an agreement forest C of Ω of order at most k . We can assume C contains a component G with labels both inside and outside of the cluster S , otherwise C can be decomposed to form an agreement forest for Ω_{above} or Ω_{below} with an order smaller than A or B respectively, contradicting their optimality. This component G can be cut at the vertex corresponding to the root of the cluster S to separate the labels in S from those outside S , we call the resulting forest C' . Now it is clear that each component in C' contains either only labels in S or outside of S , making it possible to separate C' into agreement forests A^* and B^* for clusters Ω'_{above} and Ω'_{below} respectively, by inserting the dummy labels at the two end of the cut made in G . We also know $\text{ord}(A^*) + \text{ord}(B^*) = \text{ord}(C') = \text{ord}(C) + 1 \leq k+1$, which immediately gives an agreement forest for Ω'_{above} or Ω'_{below} of smaller order than A' or B' , contradicting their optimality. ◀

For both types of cluster reduction it can be the case that performing the reduction increases the runtime of a depth bounded search algorithm if the clusters are not evenly distributed, because one might need to solve a cluster to optimality instead of only solving $\text{MAF}(\Omega, k)$ for a single parameter k . Only considering somewhat balanced clusters (e.g., in terms of number of leaves) might help prevent this; additionally, in practice a good use of a lower bound algorithm to terminate earlier or a heuristic algorithm to find a good upper bound for the clusters can reduce the number of parameters k' for which $\text{MAF}(\Omega, k')$ needs to be determined.

References

- 1 Steven Kelk. A pedagogical implementation of the cluster reduction, 2026. URL: <https://skelk.sdf-eu.org/cluster/>.
- 2 Charles Semple, Mike Steel, et al. *Phylogenetics*, volume 24. Oxford University Press on Demand, 2003.
- 3 Feng Shi, Jianer Chen, Qilong Feng, and Jianxin Wang. A parameterized algorithm for the maximum agreement forest problem on multiple rooted multifurcating trees. *Journal of Computer and System Sciences*, 97:28–44, 2018. doi:10.1016/j.jcss.2018.03.002.
- 4 Yufeng Wu. A practical method for exact computation of subtree prune and regraft distance. *Bioinformatics*, 25(2):190–196, 2009. doi:10.1093/bioinformatics/btn606.